SAVEETHA SCHOOL OF ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Department of Computer Science and Engineering

ASSIGNMENT 1

CSA5112 – Cryptography and Network Security

Secure Patient-Record Exchange in a Multi-Hospital Network

Student information	**Details**
Name	Sathish M
Register Number	192424358
Course Code	CSA5112
Course Name	Cryptography and Network Security
Assignment	Assignment 1

CO1: Apply classical encryption techniques and access-control principles to examine the protection of information.

Scope: The following work answers the twelve tasks in the supplied assignment brief. The explanations, examples and reference implementation have been newly prepared for this submission. Classical algorithms are evaluated as academic exercises, not recommended for real patient records.

System description and assumptions

The network has one central hospital and 25 clinics. A clinic may send a patient's identity, diagnosis, test results, prescription and billing details. The public network is outside the healthcare organisation's control. The main security concern is not simply that an outsider might read a message: a changed dosage, a false doctor's order or a repeated prescription could harm a patient.

For alphabetic examples, spaces and punctuation are removed and A=0 through Z=25. For the identifier exercise, the defined alphabet is 0–9 followed by A–Z, giving 36 symbols. Padding is marked and the original message length is retained so genuine trailing characters are not accidentally discarded.

Risk values use a 1–5 scale for likelihood and impact. The score is their product. These are qualitative planning estimates, not measured incident rates.

TASK 1 – Threats, Attacks and Risks (4 Marks)

A threat is an actor or event capable of causing damage. A vulnerability is a weakness that makes the damage possible. An attack is the actual attempt to exploit that weakness. Risk describes the expected seriousness of the outcome, combining likelihood and impact. In this scenario, an authorised but dishonest employee is a threat; a key shared across clinics is a vulnerability; copying ciphertexts is an attack; and later disclosure of diagnoses is a risk.

Attack scenario	**Passive / active**	**Likelihood**	**Impact**	**Risk**	**Reason**
Changing a medicine or dosage before delivery	Active	4	5	20	An altered instruction can lead to an incorrect treatment.
Re-sending a previously accepted prescription	Active	4	5	20	A repeated order can cause duplicate administration.
Listening to clinic-to-hospital traffic	Passive	4	5	20	Sensitive diagnosis and identity details may be exposed.
A clinic insider collecting encrypted messages	Passive	4	4	16	A large sample and predictable formats assist cryptanalysis.
Sending an order while pretending to be a specialist	Active	3	5	15	Staff may act on an unauthorised clinical instruction.
Flooding or disabling the	Active	3	4	12	Treatment may be delayed

records service					because records are unavailable.
Observing message size, timing and frequency	Passive	4	3	12	An observer may infer clinic activity or urgent referrals.

The first response should address prescription alteration and replay. Both have the highest patient-safety impact. A receiver should verify the prescriber's identity, check that the prescription has not changed, and reject an order identifier that has already been processed. Confidentiality remains important, but a perfectly secret message can still be dangerous if it is forged or replayed.

TASK 2 – Security Goals for Each Healthcare Function (3 Marks)

Function	Most important goals	Reason
Uploading a patient record	Confidentiality, integrity, authentication	The record must remain private, arrive unchanged and be associated with the correct clinic.
Sending a prescription	Integrity, authentication, availability	The medicine and dose must be correct, the prescriber must be genuine, and the instruction must arrive in time.
Specialist consultation	Confidentiality, authentication, integrity	Sensitive advice must be seen by the treatment team and attributable to the specialist who issued it.

Classical encryption primarily changes the appearance of a message. It can provide limited concealment against casual observation, but it does not prove that a message is authentic or unchanged. Availability is also outside the purpose of a cipher.

For example, a shift cipher can be modified by changing a ciphertext character. Decryption still produces a character at that position; the receiver has no built-in indication that an attacker changed it. Similarly, if 25 clinics share a key, a valid ciphertext does not identify which clinic or employee created it. Authentication, integrity checking, duplicate detection and audit records are separate requirements.

TASK 3 – Bell-LaPadula Access-Control Model (5 Marks)

Bell-LaPadula (BLP) is a confidentiality model. Its simple-security rule prevents a subject from reading above its clearance (“no read up”). Its *-property prevents a subject from writing information to a lower classification (“no write down”). A compartment adds a need-to-know condition: a matching level alone is not enough.

Classification	Assigned role	Illustrative compartments
Top Secret (4)	Medical Director	GENERAL, CARDIO, LAB, BILLING
Secret (3)	Specialist doctor	GENERAL, CARDIO, LAB
Confidential (2)	Nurse / lab employee	GENERAL, LAB

Restricted (1)	Billing / front desk	GENERAL, BILLING
Public (0)	Patient	GENERAL

Object	**Classification**	**Compartment**
Public clinic information	Public	GENERAL
Invoice and payment record	Restricted	BILLING
Routine laboratory result	Confidential	LAB
Nursing progress note	Confidential	GENERAL
Specialist cardiology report	Secret	CARDIO
Sensitive allergy dossier	Secret	GENERAL
Executive incident file	Top Secret	GENERAL

Request	**Decision**	**BLP reasoning**
Medical Director reads executive incident file	Allow	Clearance is equal to the object level and GENERAL is available.
Specialist reads cardiology report	Allow	Secret clearance and CARDIO compartment match.
Nurse reads a laboratory result	Allow	Confidential clearance includes LAB.
Billing employee reads an invoice	Allow	Restricted clearance includes BILLING.
Patient reads public clinic guidance	Allow	Public information is at the patient's level.
Patient reads the Secret allergy dossier	Deny	The patient would be reading above Public.
Billing employee reads the cardiology report	Deny	The employee lacks both the level and CARDIO compartment.
Nurse reads the Secret cardiology report	Deny	Confidential is below Secret and CARDIO is not assigned.
Specialist writes a Secret diagnosis into a Public file	Deny	This would send high-level information downward.
Medical Director writes a clinical narrative directly into a Restricted invoice	Deny	The *-property blocks a high-to-low write.
Nurse writes a confidential clinical note into a Restricted invoice	Deny	The target is below the nurse's current level.
Nurse requests a Secret allergy record during an emergency	Deny normally; controlled override only	A break-glass procedure is an exception, not a normal BLP permission.
Medical Director approves a report for lower classification	Controlled exception	A trusted declassifier must record scope, reason and reviewer.

Specialist sends billing codes to a Restricted file	Through trusted filter	A trusted process releases only approved billing fields, not the diagnosis.

A nurse may need a high-level allergy fact to avoid administering a dangerous medicine. Strict BLP denies the request. A practical healthcare design needs emergency access limited to one patient and a short time, with a reason entered and a later review. A safer design may also expose a minimal allergy alert at the level required by the treating nurse.

A specialist should not copy a complete clinical note into a billing record. A trusted downgrading process can release only the billing code and amount. Tranquility means classifications should not be casually altered while the system operates. A trusted subject is a carefully controlled person or process permitted to perform an exceptional operation such as declassification.

BLP does not guarantee data correctness, authenticate users, prevent all covert channels or keep the service available. It is a useful confidentiality policy but is not sufficient as the sole healthcare control.

TASK 4 – Classical Cipher Design (4 Marks)

Caesar shift

Let k be an integer from 1 to 25. Encryption is $C=(P+k) \bmod 26$ and decryption is $P=(C-k) \bmod 26$. For the example PATIENTRECORD, k=4 produces TEXMIRVIGSVH. The receiver subtracts four from each ciphertext value to recover the original.

Vigenère cipher

A keyword supplies a different shift at each position. Encryption is $C_i=(P_i+K_i) \bmod 26$ and decryption is $P_i=(C_i-K_i) \bmod 26$. With keyword CARE and plaintext PATIENT, the repeated key is CARECAR. This avoids one fixed shift for every character, but repeating the keyword creates a period.

Columnar transposition

Write the plaintext into rows under a keyword, then read columns in alphabetical order of the keyword letters. For key HOSP, the column order is determined by sorting H, O, S and P. Decryption puts the ciphertext back into those columns and reads across rows. This method changes positions rather than the letters themselves.

These three methods are meaningfully different: Caesar substitutes with one shift, Vigenère uses a repeating sequence of shifts, and columnar transposition rearranges positions. They are suitable for comparing classical design ideas, not for protecting real medical messages.

TASK 5 – Key-Space and Mathematical Analysis (5 Marks)

Method	**Key-space**	**Mathematical description**	**Security observation**
Caesar	25 effective keys	$C=(P+k) \bmod 26$	Every key can be tested quickly.
Monoalphabetic substitution	26! possible mappings	$C=\pi(P)$	The large count does not remove language-frequency clues.

Vigenère with m key letters	26^m strings	$C_i=(P_i+K_i) \bmod 26$	A repeated key creates a detectable period.
Columnar transposition with n columns	n! column orders	Column positions are permuted	Letter frequencies remain exactly the same.
Hill n×n over mod 26	Invertible matrices only	C=KP mod 26	A valid inverse exists only if gcd(det K,26)=1.

For a 2×2 Hill example, take K=[[3,3],[2,5]]. Its determinant is 3×5−3×2=9. Since gcd(9,26)=1, the matrix is invertible modulo 26. The inverse of 9 modulo 26 is 3 because 9×3=27≡1. The adjugate matrix is [[5,−3],[−2,3]]. Multiplying by 3 and reducing modulo 26 gives K^{-1}=[[15,17],[20,9]].

A large theoretical key-space is not proof of security. A monoalphabetic substitution uses one fixed replacement for each letter. In a long clinical note, repeated letters, common words and digrams still provide clues. An attacker can use language statistics rather than enumerate all 26! keys.

TASK 6 – Cryptanalytic Resistance (4 Marks)

Cipher	**Attack method**	**Expected weakness**
Caesar	Exhaustive shift search and frequency analysis	Only 25 shifts exist; one is quickly identified.
Monoalphabetic	Letter frequency, digrams and word patterns	The substitution preserves the shape of the frequency distribution.
Vigenère	Kasiski examination and index of coincidence	Repeated keyword length can be estimated; columns can then be analysed separately.
Columnar transposition	Try column orders and score common digrams	The character frequencies are unchanged, so language statistics remain useful.
Hill	Known-plaintext linear algebra	Sufficient independent plaintext/ciphertext blocks reveal the linear mapping.

The index of coincidence is $IC=\Sigma f_i(f_i-1)/(N(N-1))$, where f_i is the count of symbol i and N is the number of symbols. Caesar and transposition preserve the symbol counts, so they preserve IC. Vigenère often reduces the overall IC because several shifts are mixed; testing candidate periods and examining each resulting column can reveal the underlying language pattern.

For ordinary reused keys, Caesar is the easiest to break. Monoalphabetic substitution and transposition both leak substantial language structure, though the practical attack differs. Reused-key Vigenère resists a simple single-letter count better but becomes vulnerable as the amount of ciphertext grows. Hill is particularly exposed when known plaintext blocks are available. No ranking makes these methods acceptable for live patient information.

TASK 7 – Transposition and Product Ciphers (3 Marks)

Transposition can break up visible word shapes by changing character positions. It does not change the total number of A's, B's or other letters. A substitution followed by transposition combines two operations and may make a short message less obvious, but the transposition still preserves the frequency distribution produced by substitution. The combined method therefore does not remove statistical leakage.

A clinic employee manually applying two keys would need to remember the substitution mapping and the column order. A small keying error can make a whole record unreadable. For rural clinics, manual multi-stage encryption is error-prone; any classroom implementation should be automated. A real service should use modern authenticated encryption with managed keys.

TASK 8 – Steganography and Medical-Image Safety (3 Marks)

In least-significant-bit (LSB) hiding, each message bit replaces the lowest bit of a selected grayscale pixel. A 256×256 image has 65,536 pixels, so a one-bit-per-pixel design has a theoretical capacity of 8,192 bytes before headers and length information are reserved.

Property	**LSB hiding**	**Encryption**
Purpose	Conceals that a payload is present.	Conceals the meaning of a message.
Capacity	Limited by the carrier image and number of modified bits.	Depends on the cipher and message format, not an image.
Visual effect	Small bit changes may be hard to see.	Does not require changing a scan.
Robustness	Weak under resizing, filtering and lossy compression.	Ciphertext survives only if transmitted without corruption; integrity must be checked separately.
Detectability	Statistical analysis may identify altered bit patterns.	The existence of encrypted communication may be apparent.

A text circular can hide bits by encoding 0 as one space between words and 1 as two spaces. It has low capacity and is easily destroyed by word processors or e-mail systems that normalise whitespace. Neither text spacing nor LSB hiding should be relied on for medical delivery.

A diagnostic image must not be altered casually. Even a small pixel change may affect quantitative measurements or machine-assisted interpretation. Keep the original image unchanged. If a demonstration is required, use a synthetic or non-diagnostic carrier. Encrypt the report before embedding it: if the hiding is discovered, the extracted payload should still be unreadable. Lossy JPEG processing can destroy LSB payload bits.

TASK 9 – Implementation and Testing (4 Marks)

The appendix contains one Python reference program for the core demonstrations. It is designed to run in Google Colab or a local Python 3 environment. It includes encryption/decryption round trips, a Hill cipher over 36 symbols, a simple cryptanalysis demonstration, access-control checks, table tampering, image transformations and LSB extraction.

The test plan uses the same generated text lengths—100, 1,000, 10,000 and 100,000 characters—for timing. It also tests digits in a patient identifier, an incomplete Hill block, a modified lab value and a row swap. For every reversible transformation, compare the recovered value with the original rather than relying on visual inspection.

Test case	**Validation rule**
Alphabetic ciphers	decrypt(encrypt(message)) must equal the normalised message.
Hill cipher mod 36	Pad the final block; retain original length; recovered characters must match.
Patient identifier	PT-20481 must retain all digits and letters after documented normalisation.
Lab table	Compare original and recovered rows; test a single-cell edit and a row swap.
Image encryption	Reverse shift/permutation and compare every pixel.
LSB hiding	Extract the stored length and verify the recovered byte sequence exactly.

A basic additive checksum S=(sum of character codes) mod 10,000 detects some changes but not row swaps because addition ignores order. A position-weighted checksum $S=\Sigma(i\times code_i)$ mod 65,521 detects many reorderings, but two compensating edits can cancel. Both are unkeyed and can be recomputed by an attacker. They are not substitutes for an authentication code.

TASK 10 – Experimental Evidence and Interpretation (3 Marks)

Measurement	**Method**	**What it tells us**
Letter-frequency graph	Count each alphabet symbol in plaintext and ciphertext.	Caesar and transposition preserve counts; Vigenère tends to flatten them.
Index of coincidence	Calculate IC for each ciphertext using the same alphabet.	Compare concentration of symbol frequencies; do not compare results from different alphabet sizes as if identical.
Execution time	Repeat encryption/decryption at the four required input sizes and report median time.	Shows implementation cost and how work grows with message length.
Attack success	Record whether the correct key or plaintext was recovered at each message length.	Shows whether more intercepted material helps the selected attack.

The expected interpretation is that Caesar is recovered by a complete shift search, while transposition leaves the plaintext frequency profile unchanged. Monoalphabetic substitution changes the labels of the peaks but not their relative sizes. Vigenère can flatten the overall histogram, yet a reused keyword leaks periodic structure. Timing is machine-dependent, so numerical results must come from the actual run and should include the computer and Python version.

Do not present invented timing values or attack percentages as measured results. The supplied program prints measurements on execution. Paste the output and generated plots into the report after running it, and state the random seed and input sizes.

TASK 11 – Final Design Recommendation (4 Marks)

For the classroom comparison, a layered classical demonstration can combine a polyalphabetic substitution with a transposition and apply BLP to the records. This illustrates that different transformations conceal different features. It does not provide reliable integrity, authentication or protection against a knowledgeable attacker.

For an actual healthcare network, do not deploy classical ciphers as the security boundary. Use authenticated encryption, unique per-message nonces, per-clinic identities, session key establishment, rotation and revocation. Every prescription should carry a unique identifier and be checked for replay. Role-based permissions should be combined with classification, audit logs and an emergency access workflow.

A hub-and-spoke key-management arrangement is more manageable than creating a separate long-term secret for every pair of clinics. A compromised clinic credential should be revocable without replacing every other clinic's credentials. The application should handle cryptographic operations automatically so staff do not need to memorise or manually enter keys.

Residual risks include compromised endpoints, stolen accounts, authorised insiders, network outages, incorrect clinical data entered before encryption and metadata leakage. Classical ciphers cannot guarantee integrity, identity, non-repudiation or availability.

TASK 12 – Day-4 Design Challenge: Insider and Hidden Reports (8 Marks)

A. Revised threat picture

The attacker now has an insider position and a long collection period. This is more serious than occasional external interception. Predictable headers provide known plaintext, and the use of one key across many messages allows observations to be combined. The assumption that every clinic keeps the same key secret is no longer defensible.

B. Test of the earlier design

For Vigenère, at each known plaintext position the key value is $K_i=(C_i-P_i) \bmod 26$. If the same keyword repeats, known headers from multiple messages expose corresponding key positions. The attacker can align the messages, estimate the period with repeated-sequence distances or IC, and then solve the key columns. Bell-LaPadula still controls normal application reads, but it cannot undo ciphertext already copied by an authorised insider.

C. Adaptation

The shared long-term key should be retired. In a classroom-only redesign, give each clinic a different key, change keys frequently, avoid predictable plaintext prefixes, and use a fresh key for each message where the exercise permits. Add length-preserving metadata inside the encrypted body rather than exposing fixed labels. These changes reduce the value of a large ciphertext collection but do not turn classical algorithms into modern security.

For real deployment, use per-clinic identities and authenticated per-session encryption, unique nonces, replay counters, central revocation, and audited access. If a short report must be hidden in an image for a demonstration, encrypt it first and use a lossless, non-diagnostic carrier. Never modify the clinical master scan.

D. Defence

The mathematical key-space calculations are useful for comparing the algorithms, but the attacks exploit structure rather than simply trying every key. The known-header equation reveals Vigenère key positions; repeated use supplies more evidence. Per-clinic credentials limit the impact of one compromised clinic, while automated key rotation reduces manual burden. A high-capacity or visually subtle LSB channel does not guarantee secrecy or robustness.

The final decision is to redesign the real service rather than retain the original shared-key arrangement. Classical techniques remain useful for demonstrating cryptographic concepts, but they cannot guarantee message authenticity, tamper resistance, non-repudiation or availability.

Conclusion

The patient-record network needs more than encrypted-looking text. It needs verified identities, tamper detection, replay prevention, carefully limited access and a dependable recovery process. The classical methods in this assignment reveal important weaknesses: small key spaces, preserved language statistics, repeated-key leakage and linear known-plaintext recovery. Bell-LaPadula helps enforce confidentiality but does not protect integrity. A modern authenticated design is required for real clinical use.

APPENDIX – Python Implementation for Google Colab

Run the following as one cell in Google Colab.

```
# CSA5112 Assignment 1 -
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <numeric>
#include <stdexcept>
#include <cctype>
#include <cstdlib>

using namespace std;

// Alphabet: A-Z followed by 0-9
const string ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
const int MOD = 36;

// Convert a character into its alphabet index
int charToNum(char c) {
    c = toupper(static_cast<unsigned char>(c));
    size_t pos = ALPHABET.find(c);

    if (pos == string::npos)
        throw invalid_argument("Invalid character in message");

    return static_cast<int>(pos);
}

char numToChar(int n) {
    return ALPHABET[n % MOD];
}

int mod(int a, int m) {
    return ((a % m) + m) % m;
}

// Remove spaces and punctuation
string cleanText(const string& text) {
    string result;

    for (char c : text) {
        if (isalnum(static_cast<unsigned char>(c)))
            result += toupper(static_cast<unsigned char>(c));
    }

    return result;
}

// ------------------------------------------------
// 1. CAESAR CIPHER
```

```
// ------------------------------------------------

string caesarEncrypt(string text, int key) {
    text = cleanText(text);
    string result;

    for (char c : text) {
        int x = c - 'A';
        result += char('A' + (x + key) % 26);
    }

    return result;
}

string caesarDecrypt(string text, int key) {
    string result;

    for (char c : text) {
        int x = c - 'A';
        result += char('A' + mod(x - key, 26));
    }

    return result;
}

// ------------------------------------------------
// 2. VIGENERE CIPHER
// ------------------------------------------------

string vigenereEncrypt(string text, string key) {
    text = cleanText(text);
    key = cleanText(key);

    if (key.empty())
        throw invalid_argument("Vigenere key cannot be empty");

    string result;

    for (int i = 0; i < (int)text.size(); i++) {
        int p = text[i] - 'A';
        int k = key[i % key.size()] - 'A';

        result += char('A' + (p + k) % 26);
    }

    return result;
}

string vigenereDecrypt(string text, string key) {
    key = cleanText(key);

    if (key.empty())
        throw invalid_argument("Vigenere key cannot be empty");
```

```
    string result;

    for (int i = 0; i < (int)text.size(); i++) {
        int c = text[i] - 'A';
        int k = key[i % key.size()] - 'A';

        result += char('A' + mod(c - k, 26));
    }

    return result;
}

// ------------------------------------------------
// 3. COLUMNAR TRANSPOSITION
// ------------------------------------------------

vector<int> getColumnOrder(const string& key) {
    vector<int> order(key.size());

    iota(order.begin(), order.end(), 0);

    stable_sort(order.begin(), order.end(),
        [&](int a, int b) {
            return key[a] < key[b];
        });

    return order;
}

string columnarEncrypt(string text, string key) {
    text = cleanText(text);

    int cols = key.size();

    if (cols == 0)
        throw invalid_argument("Column key cannot be empty");

    int rows = (text.size() + cols - 1) / cols;

    while (text.size() < rows * cols)
        text += 'X';

    vector<int> order = getColumnOrder(key);
    string result;

    for (int col : order) {
        for (int row = 0; row < rows; row++) {
            result += text[row * cols + col];
        }
    }

    return result;
}
```

```
string columnarDecrypt(string cipher, string key, int originalLength) {
    int cols = key.size();

    if (cols == 0)
        throw invalid_argument("Column key cannot be empty");

    int rows = cipher.size() / cols;

    if (rows * cols != (int)cipher.size())
        throw invalid_argument("Invalid ciphertext length");

    vector<int> order = getColumnOrder(key);
    vector<string> columns(cols);
    int position = 0;

    for (int col : order) {
        for (int row = 0; row < rows; row++) {
            columns[col] += cipher[position++];
        }
    }

    string result;

    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            result += columns[col][row];
        }
    }

    result.resize(originalLength);
    return result;
}

// ------------------------------------------------
// 4. HILL CIPHER USING MODULO 36
// Key matrix: [1 2]
//             [3 5]
// ------------------------------------------------

int inverseMod(int a, int m) {
    a = mod(a, m);

    for (int x = 1; x < m; x++) {
        if ((a * x) % m == 1)
            return x;
    }

    throw invalid_argument("Matrix is not invertible");
}

string hillEncrypt(string text) {
    text = cleanText(text);

    if (text.size() % 2 != 0)
```

```
        text += 'X';

    string result;

    for (int i = 0; i < (int)text.size(); i += 2) {
        int x = charToNum(text[i]);
        int y = charToNum(text[i + 1]);

        int c1 = mod(x + 2 * y, MOD);
        int c2 = mod(3 * x + 5 * y, MOD);

        result += numToChar(c1);
        result += numToChar(c2);
    }

    return result;
}

string hillDecrypt(string cipher, int originalLength) {
    if (cipher.size() % 2 != 0)
        throw invalid_argument("Invalid Hill ciphertext");

    // Inverse matrix modulo 36:
    // [31  2]
    // [ 3 35]

    string result;

    for (int i = 0; i < (int)cipher.size(); i += 2) {
        int x = charToNum(cipher[i]);
        int y = charToNum(cipher[i + 1]);

        int p1 = mod(31 * x + 2 * y, MOD);
        int p2 = mod(3 * x + 35 * y, MOD);

        result += numToChar(p1);
        result += numToChar(p2);
    }

    result.resize(originalLength);
    return result;
}

// ------------------------------------------------
// 5. BELL-LAPADULA ACCESS CONTROL
// ------------------------------------------------

bool blpAccess(
    int subjectLevel,
    int objectLevel,
    bool sameCompartment,
    string operation
) {
    if (!sameCompartment)
```

```
        return false;

    if (operation == "READ") {
        return subjectLevel >= objectLevel;
    }

    if (operation == "WRITE") {
        return subjectLevel <= objectLevel;
    }

    return false;
}

// ------------------------------------------------
// 6. CHECKSUM
// ------------------------------------------------

int additiveChecksum(const string& text) {
    int sum = 0;

    for (unsigned char c : text)
        sum = (sum + c) % 10000;

    return sum;
}

int weightedChecksum(const string& text) {
    long long sum = 0;

    for (int i = 0; i < (int)text.size(); i++) {
        sum += 1LL * (i + 1) *
               static_cast<unsigned char>(text[i]);
    }

    return sum % 65521;
}

// ------------------------------------------------
// 7. IMAGE PIXEL TRANSFORMATION
// Demonstration using a pixel array
// ------------------------------------------------

vector<int> shiftPixels(const vector<int>& pixels, int shift) {
    vector<int> result;

    for (int p : pixels)
        result.push_back((p + shift) % 256);

    return result;
}

vector<int> reverseShiftPixels(
    const vector<int>& pixels,
    int shift
```

```
) {
    vector<int> result;

    for (int p : pixels)
        result.push_back(mod(p - shift, 256));

    return result;
}

// ------------------------------------------------
// 8. LSB STEGANOGRAPHY
// Hides and extracts text in pixel bytes
// ------------------------------------------------

vector<unsigned char> embedLSB(
    vector<unsigned char> pixels,
    const string& message
) {
    // Store message length in the first 32 bits
    unsigned int length = message.size();

    string bits;

    for (int i = 31; i >= 0; i--)
        bits += ((length >> i) & 1) ? '1' : '0';

    for (unsigned char c : message) {
        for (int i = 7; i >= 0; i--)
            bits += ((c >> i) & 1) ? '1' : '0';
    }

    if (bits.size() > pixels.size())
        throw runtime_error("Insufficient image capacity");

    for (int i = 0; i < (int)bits.size(); i++) {
        pixels[i] =
            (pixels[i] & 0xFE) | (bits[i] - '0');
    }

    return pixels;
}

string extractLSB(const vector<unsigned char>& pixels) {
    if (pixels.size() < 32)
        throw runtime_error("Image is too small");

    unsigned int length = 0;

    for (int i = 0; i < 32; i++)
        length = (length << 1) | (pixels[i] & 1);

    if (32ULL + 8ULL * length > pixels.size())
        throw runtime_error("Invalid payload length");
```

```
    string result;

    for (unsigned int i = 0; i < length; i++) {
        unsigned char c = 0;

        for (int j = 0; j < 8; j++) {
            int bit = pixels[32 + i * 8 + j] & 1;
            c = (c << 1) | bit;
        }

        result += static_cast<char>(c);
    }

    return result;
}

// ------------------------------------------------
// MAIN FUNCTION
// ------------------------------------------------

int main() {
    try {
        string message = "PATIENT RECORD";

        cout << "CRYPTOGRAPHY AND NETWORK SECURITY\n";
        cout << "================================\n\n";

        // Caesar
        string caesar = caesarEncrypt(message, 4);
        cout << "1. CAESAR CIPHER\n";
        cout << "Encrypted: " << caesar << endl;
        cout << "Decrypted: " << caesarDecrypt(caesar, 4) << "\n\n";

        // Vigenere
        string vigenere = vigenereEncrypt(message, "CARE");
        cout << "2. VIGENERE CIPHER\n";
        cout << "Encrypted: " << vigenere << endl;
        cout << "Decrypted: " << vigenereDecrypt(vigenere, "CARE") << "\n\n";

        // Columnar transposition
        string trans = columnarEncrypt(message, "HOSP");
        cout << "3. COLUMNAR TRANSPOSITION\n";
        cout << "Encrypted: " << trans << endl;
        cout << "Decrypted: "
             << columnarDecrypt(trans, "HOSP", cleanText(message).size())
             << "\n\n";

        // Hill cipher
        string hillMessage = "PT20481";
        string hill = hillEncrypt(hillMessage);

        cout << "4. HILL CIPHER (MOD 36)\n";
        cout << "Encrypted: " << hill << endl;
        cout << "Decrypted: "
```

```
     << hillDecrypt(hill, cleanText(hillMessage).size())
     << "\n\n";

// Bell-LaPadula
cout << "5. BELL-LAPADULA ACCESS CONTROL\n";

cout << "Nurse reading Secret record: "
     << (blpAccess(2, 3, true, "READ") ? "Allowed" : "Denied")
     << endl;

cout << "Specialist reading Secret record: "
     << (blpAccess(3, 3, true, "READ") ? "Allowed" : "Denied")
     << endl;

cout << "Specialist writing to Public record: "
     << (blpAccess(3, 0, true, "WRITE") ? "Allowed" : "Denied")
     << "\n\n";

// Checksum
string record = "PT20481,GLUCOSE,98";

cout << "6. CHECKSUM\n";
cout << "Additive checksum: "
     << additiveChecksum(record) << endl;

cout << "Weighted checksum: "
     << weightedChecksum(record) << "\n\n";

// Image pixel transformation
vector<int> pixels = {100, 150, 200, 250};

vector<int> encryptedPixels = shiftPixels(pixels, 50);
vector<int> decryptedPixels =
    reverseShiftPixels(encryptedPixels, 50);

cout << "7. IMAGE PIXEL TRANSFORMATION\n";
cout << "Original pixels: ";

for (int p : pixels) cout << p << " ";

cout << "\nEncrypted pixels: ";

for (int p : encryptedPixels) cout << p << " ";

cout << "\nRecovered pixels: ";

for (int p : decryptedPixels) cout << p << " ";

cout << "\n\n";

// LSB steganography
vector<unsigned char> image(5000, 128);

string secretMessage = "PATIENT REPORT";
```

```
        vector<unsigned char> stegoImage =
            embedLSB(image, secretMessage);

        cout << "8. LSB STEGANOGRAPHY\n";
        cout << "Original message: " << secretMessage << endl;
        cout << "Extracted message: "
             << extractLSB(stegoImage) << "\n\n";

        cout << "All demonstrations completed successfully.\n";
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
        return 1;
    }

    return 0;
}
```

OUTPUT:

```
CRYPTOGRAPHY AND NETWORK SECURITY
==================================

1. CAESAR CIPHER
Encrypted: TEXMIRXVIGSVH
Decrypted: PATIENTRECORD

2. VIGENERE CIPHER
Encrypted: RAKMGNKVGCFVF
Decrypted: PATIENTRECORD

3. COLUMNAR TRANSPOSITION
Encrypted: PEEDANCXIRRXTTOX
Decrypted: PATIENTRECORD

4. HILL CIPHER (MOD 36)
Encrypted: R6I80IBQ
Decrypted: PT20481

5. BELL-LAPADULA ACCESS CONTROL
Nurse reading Secret record: Denied
Specialist reading Secret record: Allowed
Specialist writing to Public record: Denied

6. CHECKSUM
Additive checksum: 1150
Weighted checksum: 10924

7. IMAGE PIXEL TRANSFORMATION
Original pixels: 100 150 200 250
Encrypted pixels: 150 200 250 44
Recovered pixels: 100 150 200 250

8. LSB STEGANOGRAPHY
Original message: PATIENT REPORT
Extracted message: PATIENT REPORT
```

```
All demonstrations completed successfully.
```